

SUMMER TRAINING COURSE

PROJECT REPORT

Course Code- CSE443

FOOD DELIVERY SYSTEM USING OBJECT-ORIENTED PROGRAMMING AND DSA IN JAVA

Submitted by

Swayam Shree Sahu

Registration Number: **12404561**

Course / Domain: Data Structures and Algorithms

Under the Guidance of

Sanskar Drolia and Rohit Rai (CipherSchools)

School of AI and Emerging Technologies

LOVELY PROFESSIONAL UNIVERSITY

PUNJAB, INDIA



L LOVELY
P PROFESSIONAL
U UNIVERSITY

DECLARATION

I hereby declare that the project work entitled "Food Delivery System using Object-Oriented Programming and Data Structures in Java" is an authentic record of my own work carried out as a requirement of the Summer Training Program 2026 in the topic of Object-Oriented Programming and Data Structures.

I further declare that this project has been completed during the training period and that the work presented in this report is based on my own learning, effort, implementation, and understanding. The information and resources used in preparing this report have been acknowledged wherever required.

I understand that any violation of academic integrity, including plagiarism or misrepresentation of work, may result in disciplinary action as per university rules.

Swayam Shree Sahu

12404561

CERTIFICATE

This is to certify that the declaration statement made by this student is correct to the best of my knowledge and belief. He has completed this Project during the Summer Training Program under mentor guidance and supervision. The present work is the result of his original investigation, effort, and study.



CERTIFICATE OF COMPELETION

This is to certify that

Swayam shree Sahu

studying at **Lovely Professional University** has successfully completed training in

Data Structures and Algorithms

organized by **CipherSchools** during the period of **August-2026**

This certificate acknowledges the recipient's dedication, commitment, and successful completion of the learning program at CipherSchools.



Scan to Verify



ANURAG MISHRA
Founder CipherSchools

Certificate ID: CSW2026-18091

ACKNOWLEDGEMENT

I would like to express my sincere gratitude and heartfelt thanks to all those who contributed to the successful completion of this project.

First and foremost, I extend my deepest appreciation to my course mentors, Sanskar Drolia and Rohit Rai, for invaluable guidance, constant encouragement, and expert supervision throughout the course of this summer training program. The feedback and code reviews provided at each stage helped me build a modular, high-performing system.

I am grateful to the School of AI and Emerging Technologies, Lovely Professional University, Phagwara, for providing the academic environment, infrastructure, and curriculum support needed to complete the training in a disciplined way.

Swayam Shree Sahu

12404561

TABLE OF CONTENTS

INTRODUCTION TO COURSE AND PROJECT	6
PROFILE OF THE PROBLEM	9
PROBLEM ANALYSIS	10
SOFTWARE REQUIREMENT ANALYSIS	11
DESIGN	12
PROJECT LEGACY	15
SOURCE CODE AND SYSTEM SNAPSHOTS	16
BIBLIOGRAPHY	32

1. INTRODUCTION

1.1. Course Overview

- Course Code: CSE443 (Summer Training)
- Domain: Data Structures & Algorithms Live
- Schedule: Online LIVE interactive lectures (Monday–Friday, 2 hours daily)
- Mentors: Sanskar Drolia, Rohit Rai

1.2. Training Content & Core Initiatives

Core DSA & Clean Software Architecture:

- Advanced Arrays, Linked Lists, Hash Tables, Binary Trees, and Graph Traversal Algorithms.
- Object-Oriented Analysis & Design (OOAD), SOLID Design Principles, and Clean Architecture (Domain-Driven Design).
- Dependency Inversion and Constructor Injection patterns.

Resume Building Session:

- Dedicated workshops on creating ATS-friendly technical resumes by highlighting essential algorithmic skills, optimizing project descriptions, and structuring layouts for recruiter visibility.

Industrial Sessions:

- Guest lectures hosted by industry leaders from top technology firms including Google, Microsoft, ISRO, Tesla, NASA, and Adobe.
- Direct insights into software deployment workflows, industry expectations, and career roadmaps.

Tech Synergy – Coding Contest:

- Reward-based competitive programming challenges testing real-world algorithmic efficiency, time complexity optimization, and data structure selection.

Top performers received recognition badges and Cipher Points.

Learn in Public Initiative: Encouraged continuous technical documentation and public sharing of daily coding achievements to foster personal branding and peer accountability.

Cipher Tank – Project Pitching Competition:

A competitive pitching showcase where students presented major projects to an expert panel. Promising builds received company referrals and project incubation support.

Program Benefits & Outcomes:

- Earned Redeemable Cipher Points.
- Lifetime access to live lecture recordings.
- Participation in the Internship Readiness Program.
- Industry-recognized Certificate of Completion upon defense.

1.3 Project Overview

QuickBites is a browser-based local food delivery system developed in Java. It demonstrates a complete food ordering workflow using a lightweight custom backend, static frontend pages, JSON REST APIs, in-memory storage, and multiple user roles. It focuses on clear Java classes, object composition, and custom data structures.

1.4 Objectives of the Study

Objective	Description
OOP application	Design a modular food delivery system using classes for users, restaurants, orders, carts, and food items.
DSA implementation	Apply queue, stack, priority queue, merge sort, and binary search in meaningful project workflows.
Multi-role access	Provide separate flows for customers, restaurants, delivery partners, and administrators.
Local web delivery	Serve HTML, CSS, JavaScript, and JSON APIs through Java's built-in <code>HttpServer</code> .

1.5 Scope of the Project

The project covers restaurant browsing, menu viewing, cart management, coupon-based checkout, order status updates, restaurant-side order processing, delivery partner assignment, and administrative monitoring. It is designed for local execution and demonstration, with all data stored in memory during runtime.

2. PROFILE OF THE PROBLEM

2.1 Problem Statement

Food delivery platforms coordinate many connected activities: customer ordering, restaurant preparation, rider assignment, delivery status tracking, and operational monitoring. The problem addressed by this project is to model a realistic food delivery workflow while applying OOP and DSA concepts in a clear and demonstrable way.

2.2 Rationale of the Study

A food delivery system is a suitable project for learning because it naturally requires entities, relationships, state changes, collections, and algorithmic decisions. Customers interact with carts and orders, restaurants manage queues, delivery partners are selected by priority, and administrators require summary data. These requirements create a practical context for applying Java collections and custom data structures.

2.3 Significance of the Project

The project strengthens understanding of encapsulation, inheritance, composition, HTTP routing, JSON handling, frontend-backend communication, and algorithm selection. It also shows how a small Java application can behave like a complete web system without using a large framework.

3. PROBLEM ANALYSIS

3.1 Product Definition

QuickBites is a local food ordering application that supports four roles. Customers search and sort restaurants, add food items to a cart, apply coupons, and place orders. Restaurants accept or reject orders and update preparation states. Delivery partners handle assigned orders, and administrators monitor system statistics and order logs.

3.2 Feasibility Analysis

Area	Feasibility
Technical	Feasible using Java 17, Maven, Gson, static frontend assets, and built-in HttpServer.
Operational	Feasible for classroom use because test users and sample data are preloaded.
Economic	Feasible because it uses open-source tools and local execution without paid services.

3.3 Project Plan

The project was developed through requirement identification, class design, data structure implementation, backend API development, frontend page creation, integration, testing, and documentation.

4. SOFTWARE REQUIREMENT ANALYSIS

4.1 Functional Requirements

Role	Main Requirements
Customer	Login, browse restaurants, search, sort, view menu, add to cart, update cart, apply coupon, place order, track status.
Restaurant	Login, view incoming orders, accept or reject orders, mark preparation and readiness states.
Delivery Partner	Login, view assigned deliveries, accept delivery, pick up order, mark order delivered.
Admin	Login, monitor total customers, restaurants, orders, active orders, delivered orders, and rider availability.

4.2 Non-Functional Requirements

Requirement	Description
Performance	Restaurant sorting uses merge sort and exact restaurant lookup can use binary search.
Usability	Separate pages are provided for customer, restaurant, cart, admin, and login workflows.
Maintainability	Code is separated into app, model, and dsa packages.
Reliability	Input checks are performed for invalid logins, missing restaurant IDs, empty carts, and invalid routes.
Portability	The application can run locally with Java 17 and Maven.

4.3 System Constraints

The system uses in-memory data and is intended for single-machine demonstration. Since no external database is used, data resets when the application restarts. The application is not connected to real payment gateways, maps, SMS services, or production delivery networks.

5. SYSTEM DESIGN

5.1 Architecture

The system follows a simple layered structure. The frontend contains HTML, CSS, and JavaScript assets. The app package contains the server launcher, router, API handlers, and in-memory datastore. The model package defines domain objects such as User, Customer, Restaurant, DeliveryPartner, Admin, FoodItem, Cart, and Order. The dsa package contains custom algorithm and data structure implementations.

Layer / Package	Responsibility
frontend	User interface pages and JavaScript API calls.
app.Main	Starts the local HTTP server on port 8080.
app.Router	Serves static files and handles JSON API routes.
app.DataStore	Stores sample users, restaurants, menus, coupons, orders, and lookup maps.
model	Defines OOP domain classes and role-specific behavior.
dsa	Provides custom queue, stack, priority queue, merge sort, binary search, and delivery assignment logic.

5.2 Key Classes

Class	Purpose
User	Base abstraction for all logged-in users.
Customer	Maintains customer details, cart, and order history stack.
Restaurant	Stores restaurant information and menu items.
DeliveryPartner	Tracks rider rating, availability, and current assignment.
Cart	Maintains selected items and total price before checkout.
Order	Stores transaction details, coupon discount, delivery partner, and status.
DataStore	Acts as an in-memory database with sample data and lookup maps.

5.3 Technology Stack

Backend	Java 17, Maven, Java built-in HttpServer
JSON Handling	Google Gson 2.10.1

Frontend	HTML5, CSS3, Bootstrap 5, JavaScript
Storage	In-memory Java collections
Build Tool	Apache Maven
Run URL	http://localhost:8080

5.4 API Implementation

The Router class handles endpoints such as login, restaurant listing, cart operations, order placement, order status updates, order retrieval, and admin statistics. Requests and responses are exchanged as JSON, allowing the frontend pages to communicate with the Java backend asynchronously.

5.5 Main Workflow

Step	System Activity
1	Customer logs in and views available restaurants.
2	Customer searches or sorts restaurants and selects menu items.
3	Cart calculates subtotal and applies coupon discount where valid.
4	Order is created, stored, pushed to customer history, and enqueued as pending.
5	Restaurant accepts, prepares, and marks the order ready.
6	Delivery manager assigns an available delivery partner by priority.
7	Delivery partner completes pickup and delivery, and admin statistics update.

5.6 DATA STRUCTURES AND ALGORITHMS USED

Data Structure / Algorithm	Project Usage	Benefit
CustomQueue	Stores pending orders waiting for restaurant action.	FIFO processing reflects real order arrival sequence.
CustomStack	Stores customer order history.	Newest orders appear first for quick review.
CustomPriorityQueue	Stores and selects available delivery partners.	Higher-rated available rider can be selected efficiently.
Merge Sort	Sorts restaurants by name, rating, or delivery speed.	Reliable $O(n \log n)$ sorting behavior.
Binary Search	Finds restaurant by name after sorted lookup.	Fast $O(\log n)$ exact search.

HashMap	Maps user IDs, usernames, and coupon codes.	Average $O(1)$ lookup for login and coupon validation.
---------	---	--

6. PROJECT LEGACY

The QuickBites Food Delivery System leaves behind a practical academic software model that demonstrates how real-world food ordering platforms can be designed using Java, Object-Oriented Programming, and Data Structures. The project is not only a working application but also a learning resource for understanding how customers, restaurants, delivery partners, and administrators interact within a digital service system.

The legacy of this project lies in its clear modular design. Each major role and feature is represented through separate classes and responsibilities, making the system easier to understand, maintain, and extend. Concepts such as inheritance, encapsulation, composition, queues, stacks, priority queues, sorting, searching, and hash maps are applied in meaningful workflows instead of being used only theoretically.

This project can be improved further by adding database connectivity, online payment support, real-time location tracking, user registration, order notifications, and enhanced security features. Future students or developers can use QuickBites as a base project and expand it into a more advanced food delivery platform.

Overall, QuickBites creates a strong foundation for future development and serves as evidence of practical learning in software design, Java programming, and data structure implementation. It reflects how classroom concepts can be converted into a functional and user-oriented application.

7. SOURCE CODE & SNAPSHOTS

// Label: Main.java

// Purpose: Starts the QuickBites local server.

```
package app;
```

```
import com.sun.net.httpserver.HttpServer;
```

```
import java.net.InetSocketAddress;
```

```
import java.util.concurrent.Executors;
```

```
public class Main {
```

```
    private static final int PORT = 8080;
```

```
    public static void main(String[] args) {
```

```
        System.out.println("QuickBites - Food Delivery System");
```

```
        try {
```

```
            Class.forName("app.DataStore");
```

```
            HttpServer server = HttpServer.create(new InetSocketAddress(PORT), 0);
```

```
            server.createContext("/", new Router());
```

```
            server.setExecutor(Executors.newCachedThreadPool());
```

```
            server.start();
```

```
            System.out.println("Server started at: http://localhost:" + PORT);
```

```
        } catch (Exception e) {
```

```
            System.err.println("Failed to start server: " + e.getMessage());
```

```
        }
```

```
    }
```

```
}
```



```
// Label: User.java  
// Purpose: Base class for Customer, Restaurant, Delivery Partner and Admin.
```

```
package model;
```

```
public abstract class User {
```

```
    private int id;
```

```
    private String username;
```

```
    private String password;
```

```
    private String role;
```

```
    public User(int id, String username, String password, String role) {
```

```
        this.id = id;
```

```
        this.username = username;
```

```
        this.password = password;
```

```
        this.role = role;
```

```
    }
```

```
    public int getId() {
```

```
        return id;
```

```
    }
```

```
    public String getUsername() {
```

```
        return username;
```

```
    }
```

```
    public String getPassword() {
```

```
        return password;
```

```
    }
```

```
    public String getRole() {
```

```
        return role;
    }
}
```

// Label: Order.java

// Purpose: Stores order details and order status.

```
package model;
```

```
import java.util.List;
```

```
public class Order {
    private int orderId;
    private int customerId;
    private int restaurantId;
    private List<Cart.CartItem> items;
    private double totalPrice;
    private String couponCode;
    private double discount;
    private double finalPrice;
    private String status;
    private int deliveryPartnerId;

    public Order(int orderId, int customerId, int restaurantId,
        List<Cart.CartItem> items, double totalPrice,
        String couponCode, double discount, double finalPrice) {
        this.orderId = orderId;
        this.customerId = customerId;
        this.restaurantId = restaurantId;
        this.items = items;
    }
}
```

```
    this.totalPrice = totalPrice;
    this.couponCode = couponCode;
    this.discount = discount;
    this.finalPrice = finalPrice;
    this.status = "PENDING";
    this.deliveryPartnerId = -1;
}
```

```
public int getOrderId() {
    return orderId;
}
```

```
public int getCustomerId() {
    return customerId;
}
```

```
public int getRestaurantId() {
    return restaurantId;
}
```

```
public String getStatus() {
    return status;
}
```

```
public void setStatus(String status) {
    this.status = status;
}
```

```
public int getDeliveryPartnerId() {
    return deliveryPartnerId;
}
```

```

    public void setDeliveryPartnerId(int deliveryPartnerId) {
        this.deliveryPartnerId = deliveryPartnerId;
    }
}

```

// Label: CustomQueue.java

// Purpose: Queue used for pending orders.

```

package dsa;

```

```

public class CustomQueue<T> {
    private static class Node<T> {
        T data;
        Node<T> next;

        Node(T data) {
            this.data = data;
        }
    }
}

```

```

    private Node<T> head;
    private Node<T> tail;
    private int size;

```

```

    public synchronized void enqueue(T item) {
        Node<T> newNode = new Node<>(item);

        if (tail == null) {
            head = tail = newNode;
        } else {

```

```

        tail.next = newNode;
        tail = newNode;
    }

    size++;
}

public synchronized T dequeue() {
    if (head == null) {
        return null;
    }

    T data = head.data;
    head = head.next;

    if (head == null) {
        tail = null;
    }

    size--;
    return data;
}

public synchronized boolean isEmpty() {
    return head == null;
}
}

// Label: CustomStack.java
// Purpose: Stack used for customer order history.

```

```

package dsa;

import java.util.ArrayList;
import java.util.List;

public class CustomStack<T> {
    private static class Node<T> {
        T data;
        Node<T> next;

        Node(T data) {
            this.data = data;
        }
    }

    private Node<T> top;

    public synchronized void push(T item) {
        Node<T> newNode = new Node<>(item);
        newNode.next = top;
        top = newNode;
    }

    public synchronized T pop() {
        if (top == null) {
            return null;
        }

        T data = top.data;
        top = top.next;
        return data;
    }
}

```

```

    }

    public synchronized List<T> toList() {
        List<T> list = new ArrayList<>();
        Node<T> current = top;

        while (current != null) {
            list.add(current.data);
            current = current.next;
        }

        return list;
    }
}

// Label: DeliveryManager.java
// Purpose: Assigns highest-rated available delivery partner.

package dsa;

import java.util.Comparator;
import java.util.List;
import model.DeliveryPartner;
import model.Order;

public class DeliveryManager {
    public static synchronized DeliveryPartner assignPartner(
        Order order, List<DeliveryPartner> partners) {

        CustomPriorityQueue<DeliveryPartner> priorityQueue =
            new CustomPriorityQueue<>(new Comparator<DeliveryPartner>() {

```

```

        public int compare(DeliveryPartner p1, DeliveryPartner p2) {
            return Double.compare(p2.getRating(), p1.getRating());
        }
    });

    for (DeliveryPartner partner : partners) {
        if (partner.isAvailable()) {
            priorityQueue.add(partner);
        }
    }

    DeliveryPartner bestPartner = priorityQueue.poll();

    if (bestPartner != null) {
        bestPartner.setAvailable(false);
        bestPartner.setCurrentOrderId(order.getId());
        order.setDeliveryPartnerId(bestPartner.getId());
        order.setStatus("READY");
    }

    return bestPartner;
}
}

```

// Label: Sort.java

// Purpose: Merge Sort for sorting restaurants.

```
package dsa;
```

```
import java.util.ArrayList;
```

```
import java.util.List;
```



```
import model.Restaurant;
```

```
public class Sort {  
    public static void mergeSort(List<Restaurant> list, String sortBy) {  
        if (list == null || list.size() < 2) {  
            return;  
        }  
  
        int mid = list.size() / 2;  
        List<Restaurant> left = new ArrayList<>(list.subList(0, mid));  
        List<Restaurant> right = new ArrayList<>(list.subList(mid, list.size()));  
  
        mergeSort(left, sortBy);  
        mergeSort(right, sortBy);  
        merge(list, left, right, sortBy);  
    }  
  
    private static void merge(List<Restaurant> list,  
                             List<Restaurant> left,  
                             List<Restaurant> right,  
                             String sortBy) {  
        int i = 0, j = 0, k = 0;  
  
        while (i < left.size() && j < right.size()) {  
            if (compare(left.get(i), right.get(j), sortBy) <= 0) {  
                list.set(k++, left.get(i++));  
            } else {  
                list.set(k++, right.get(j++));  
            }  
        }  
    }  
}
```

```

        while (i < left.size()) {
            list.set(k++, left.get(i++));
        }

        while (j < right.size()) {
            list.set(k++, right.get(j++));
        }
    }

    private static int compare(Restaurant r1, Restaurant r2, String sortBy) {
        if ("rating".equalsIgnoreCase(sortBy)) {
            return Double.compare(r2.getRating(), r1.getRating());
        } else if ("deliveryTime".equalsIgnoreCase(sortBy)) {
            return Integer.compare(r1.getDeliveryTime(), r2.getDeliveryTime());
        } else {
            return r1.getName().compareToIgnoreCase(r2.getName());
        }
    }
}

```

// Label: Search.java

// Purpose: Binary Search for finding restaurants by name.

```
package dsa;
```

```
import java.util.ArrayList;
```

```
import java.util.List;
```

```
import model.Restaurant;
```

```
public class Search {
```

```
    public static Restaurant binarySearchByName(
```

```

    List<Restaurant> list, String targetName) {

    List<Restaurant> sortedList = new ArrayList<>(list);
    Sort.mergeSort(sortedList, "name");

    int low = 0;
    int high = sortedList.size() - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;
        Restaurant midRestaurant = sortedList.get(mid);

        int cmp = midRestaurant.getName()
            .compareToIgnoreCase(targetName.trim());

        if (cmp == 0) {
            return midRestaurant;
        } else if (cmp < 0) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }

    return null;
}
}

```

QuickBites Restaurants

Login

Welcome Back

Access your QuickBites portal using your role credentials

CustomerStoreRiderAdmin

Username

e.g. cust1, rest1, delivery1, admin

Password

Sign In

Presentation Tip: Use the default accounts pre-populated:

- Customer: `cust1` (password: `password`)
- Restaurant: `rest1` (password: `password`)
- Delivery: `delivery1` (password: `password`)
- Admin: `admin` (password: `password`)

© 2026 QuickBites. College Project.

QuickBites Restaurants

Login

Delicious Food, Delivered Fast

Order from your local favourites and track your delivery in real-time. Powered by custom optimized sorting and search algorithms.

Search for your favourite restaurant...

Search

Browse Restaurants

Sort By: Name (A-Z)

Pizza

Pizza Palazzo

Italian Cuisine

★ 4.7

🕒 25 mins

View Menu

Burger

Burger Bastion

American Cuisine

★ 4.4

🕒 20 mins

View Menu

Sushi

Sushi Samurai

Japanese Cuisine

★ 4.8

🕒 35 mins

View Menu

28

Customer

RestaurantsMenu

Pizza Palazzo

Italian Cuisine★ 4.7🕒 25 mins Delivery Time

Our Menu

Margherita Pizza

Classic tomato, fresh mozzarella, basil

Mains

Add to Cart

\$12.99

Garlic Bread

Toasted baguette with garlic herb butter

Starters

Add to Cart

\$5.99

Tiramisu

Classic coffee-flavoured Italian dessert

Desserts

Add to Cart

\$6.99

Pepperoni Passion

Crispy pepperoni, cheese, tomato sauce

Mains

Add to Cart

\$14.99

Fettuccine Alfredo

Creamy alfredo sauce over fettuccine

Mains

Add to Cart

\$16.99

QuickBites

BrowseMy OrdersCart2

Hi, Alice Smith (CUSTOMER)Logout

Shopping Cart

Margherita Pizza

\$12.99 each

-

1

+

\$12.99

✕ Remove

Pepperoni Passion

\$14.99 each

-

1

+

\$14.99

✕ Remove

Order Summary

Promo Coupon Code

e.g. QUICK10, EATS20, BITE50

Apply

Available coupons: [QUICK10](#) (10%), [EATS20](#) (20%), [BITE50](#) (50% max \$15).

Subtotal

\$27.98

Discount Applied

-\$0.00

Total Price

\$27.98

Place Order

Restaurant

QuickBites

Store Dashboard

Hi, Pizza Palazzo (RESTAURANT)

Logout

RESTAURANT Dashboard

Incoming Pending Queue

Order #10

12:03:09 AM

Pepperoni Passion x1

Accept

Reject

In-Preparation / Ready Orders

No active items in kitchen.

Delivery

QuickBites

Deliveries

Hi, Dave Miller (DELIVERY)

Logout

DELIVERY Dashboard

Rider Status

Name: Dave Miller

Rating: ★ 4.8

Availability: Available

Assigned Orders

No active deliveries assigned to you at the moment.

Admin

QuickBitesAdmin Console

Hi, admin (ADMIN)

Logout

Admin Control Center

LIVE ANALYTICS

System Statistics

5RESTAURANTS

10CUSTOMERS

10TOTAL ORDERS

5ACTIVE ORDERS

5DELIVERED ORDERS

4AVAILABLE RIDERS

All System Orders Log

Order ID	Customer ID	Restaurant ID	Items Ordered	Final Cost	Status
#10	Cust #5	Rest #101	Pepperoni Passion (1)	\$14.99	PENDING
#9	Cust #4	Rest #105	Chicken Biryani (1)	\$13.59	READY
#8	Cust #3	Rest #104	Chicken Quesadilla (2)	\$19.98	PREPARING
#7	Cust #2	Rest #103	Salmon Roll (2)	\$15.98	ACCEPTED
#6	Cust #1	Rest #102	Bacon BBQ Burger (1)	\$11.99	PENDING
#5	Cust #5	Rest #105	Butter Chicken (1), Garlic Naan (2)	\$19.77	DELIVERED
#4	Cust #4	Rest #104	Burrito Bowl (2)	\$21.98	DELIVERED

8.Bibliography

1. Oracle. *Java Documentation*. Available at: <https://docs.oracle.com/en/java/>
2. Oracle. *Java Collections Framework Documentation*. Available at: <https://docs.oracle.com/javase/8/docs/technotes/guides/collections/>
3. Oracle. *Java HTTP Server API Documentation*. Available at: <https://docs.oracle.com/javase/8/docs/jre/api/net/httpserver/spec/com/sun/net/httpserver/HttpServer.html>
4. Maven Apache. *Apache Maven Project Documentation*. Available at: <https://maven.apache.org/guides/>
5. MDN Web Docs. *HTML, CSS, and JavaScript Documentation*. Available at: <https://developer.mozilla.org/>
6. GeeksforGeeks. *Data Structures and Algorithms Tutorials*. Available at: <https://www.geeksforgeeks.org/data-structures/>
7. W3Schools. *HTML, CSS, JavaScript Reference*. Available at: <https://www.w3schools.com/>
8. GitHub: https://github.com/Swayam-49/DSA_FOOD_DELIVERY_SYSTEM_SWAYAM